Eight Verification Conditions for MRDTs:
a mechanized, corrected soundness metatheory for RA-linearizability under three-way merge, with nine production data types discharged

Sal metatheory note
FP Launchpad, IIT Madras

July 3, 2026

Abstract

Mergeable replicated data types (MRDTs) resolve concurrent divergence by a *three-way* merge $\mathsf{mergeL}(l, a, b)$: the two divergent states a, b are reconciled relative to their lowest common ancestor l in a git-like version DAG. The Neem methodology reduces RA-linearizability of an MRDT to a fixed catalogue of per-data-type verification conditions (VCs), tied together by a once-and-for-all soundness meta-theorem. Mechanizing that meta-theorem in Lean 4, we find that its central induction is unsound as written — a reachable, fully LCA-legal execution of a legal add-wins MRDT defeats every bottom-up peel the proof can make — that the paper's own LCA lemma, while true, has a gap in its proof, and that no repair can lean on either of the two obvious crutches: the LCA argument cannot be forgotten (the counter MRDT provably admits no binary, CRDT-style collapse), and no lattice-flavoured contract is available (no idempotence, no usable order). What works is a *delta* discipline: three unconditional relational VCs on rc, one merge symmetry, three *feasibility-conditioned* redistribution laws, and a single contextual *causal-delta* equation — eight VCs in all — from which every reachable configuration of the replicated store is RA-linearizable, *per version*. All of it is mechanized in Lean 4 with zero `sorry`s, and the VC set is validated in the strongest currency available: nine of the twelve production MRDTs of the Sal repository — including OR-sets, an enable-wins flag, counters, a tombstone RGA and Peritext — carry kernel-checked end-to-end RA-linearizability theorems through it.

Contents

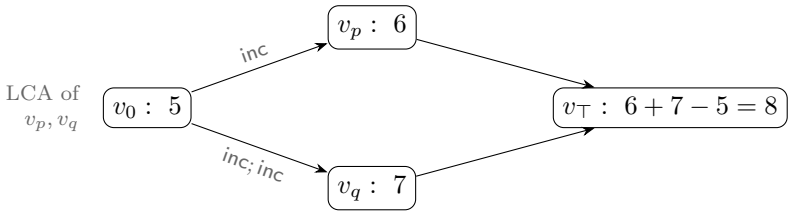

Figure 1: The counter MRDT. Both branches inherit the 5 from the fork point; the LCA argument lets the merge count each branch's *delta* exactly once. No binary merge of 6 and 7 alone can recover 8: the information "how much is shared" lives in the DAG, and the LCA argument is how the data type reads it.

1 Introduction

An MRDT execution looks like a git history: replicas fork, apply operations locally, and merge. Unlike a state-based CRDT, whose binary merge must reconcile two states with no memory of their common past, an MRDT's merge receives a third argument — the state at the *lowest common ancestor* (LCA) of the two versions being merged — and may use it to compute *deltas*: what each side did since the fork. The paradigmatic example is the counter,

$$\mathsf{mergeL}(l, a, b) \;=\; a + b - l,$$

which adds the two sides' increments-since-l instead of double-counting their common past (Figure 1).

The Neem methodology (OOPSLA 2025) promises a clean division of labour: the data-type designer discharges a fixed catalogue of equations over do, mergeL and the conflict-resolution relation rc — mostly by SMT — and a soundness meta-theorem, proved once, converts those VCs into *RA-linearizability* of every reachable configuration. This note documents what a Lean 4 mechanization of that meta-theorem, in the full **ternary, version-DAG** setting, found: the meta-theorem's central induction is unsound as written, and for LCA-sensitive data types several of the catalogue's merge VCs are false in principle when quantified over all states. It also develops the corrected metatheory: a set-relative linearization order, canonical states $\sigma(E)$ (each event set determines a unique state), and a ternary *Join Lemma* whose induction consumes a small, contextual VC surface. Its contributions:

1. **The LCA argument is load-bearing** (§3.1): MRDT verification does not reduce to CRDT verification. The counter satisfies the ternary shared-peel law but *provably violates* its binary counterpart under every LCA-forgetting collapse, so no binary metatheory can host it. "A CRDT is an MRDT that ignores its LCA" is a one-way instantiation, not an equivalence: the metatheory must be built natively ternary.

2. **The paper's LCA lemma is true but its proof has a gap** (§3.2): $L(v_\ell) = L(v_1) \cap L(v_2)$ holds, but only as a *reachability invariant* of the store, whose proof needs a generator-uniqueness induction the paper silently assumes. We mechanize the transition system and the invariant.

3. **The soundness proof has a fully LCA-legal defeater** (§3.3): a reachable configuration of a legal add-wins MRDT — its final merge sitting at an honest LCA — on which every bottom-up peel of the paper's derivation rules is stuck. The meta-theorem must be rebuilt, not patched.

4. **No lattice contract survives; a delta contract does** (§5): MRDT merges are not idempotent ($\mathsf{mergeL}(l, a, a) = 2a - l$ for the counter), the induced orders degenerate, and the honest associativity law has *four distinct LCAs* and holds only on feasible tuples. What replaces the lattice laws is a two-law *delta contract* — redistribution identities in which the LCA slot does, by arithmetic, the job idempotence does order-theoretically for CRDTs.

5. **Eight VCs suffice** (§6–§7): three relational VCs on rc (one of them carrying a different-replica guard — same-replica pairs are ordered by program order, and demanding rc cover them is unsatisfiable for real data types), merge symmetry, three feasibility-conditioned delta laws, and one contextual *causal-delta* equation. The adequacy theorem is per *version*: every version in the store, LCAs included, linearizes.

6. **The VC set is validated on the production catalogue** (§8): nine of the twelve MRDTs shipped in Sal — OR-set, an optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, a tombstone RGA, and Peritext — are discharged end-to-end, kernel-checked. The sweep produces an empirical *class map* of which data types need which strength of contract, with two surprises: tombstones are a metatheoretic trivializer, and commutativity class and delta class are independent axes.

Everything stated as a theorem below is mechanized with zero `sorry`s; §10 maps statements to Lean names and files.

2 The ternary setting

This section fixes the model: the data-type signature, the replicated store and its transitions, and the specification the metatheorem must deliver. The definitions are the paper's, with one correction flagged where it occurs.

Signature. An MRDT is $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc} \rangle$: a state space with initial state, an update function applied to *events* $e = (t, r, o)$ (a globally fresh timestamp, an origin replica, an operation), a ternary merge, and a conflict-resolution relation rc on operations used to order concurrent non-commuting pairs. We write $e(\sigma)$ for $\mathsf{do}(\sigma, e)$, and $e_1 \rightleftarrows e_2$ when $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ for every σ.

Executions. A configuration carries a *store*: a DAG of versions, each holding a state and the set of events that produced it, with per-replica heads. Transitions fork a replica, apply a fresh event at a head, query, or **merge**: pick heads v_1, v_2, find their LCA v_ℓ in the DAG, and install

a new version with state $\mathsf{mergeL}(\sigma_{v_\ell}, \sigma_{v_1}, \sigma_{v_2})$ and event set $L(v_1) \cup L(v_2)$. Visibility $\xrightarrow{\mathsf{vis}}$ records what each event's issuing replica had seen.

RA-linearizability, per version. Write $e_1 \parallel e_2$ when neither $e_1 \xrightarrow{\mathsf{vis}} e_2$ nor $e_2 \xrightarrow{\mathsf{vis}} e_1$ (concurrency).

Definition 2.1 (linearization order, set-relative)**.** *For an event set E, the order lo^E puts e_1 before e_2 iff*

$$\left(e_1 \xrightarrow{\mathsf{vis}} e_2 \ \wedge\ e_1 \not\xrightarrow{\mathsf{vis}} e_2\right) \ \vee\ \left(e_1 \parallel e_2 \ \wedge\ e_1 \xrightarrow{\mathsf{rc}} e_2 \ \wedge\ \neg\, \exists e_3 \in E.\ e_2 \xrightarrow{\mathsf{vis}} e_3 \wedge e_2 \not\xrightarrow{\mathsf{vis}} e_3\right) :$$

a visible non-commuting pair is ordered by visibility; a concurrent pair whose conflict rc *resolves is ordered by* rc *— unless e_2 is already absorbed (overwritten) by a later non-commuting event of E. The paper's order* lo *is the variant whose absorber clause ranges over the whole configuration; since a* lo*-edge asserts no absorber anywhere, $\mathsf{lo} \subseteq \mathsf{lo}^E$ pointwise, for every E.*

Definition 2.2 (RA-linearizability)**.** *A list $\pi = e^1 e^2 \cdots e^n$ witnesses a version holding state s and event set E when (i) π enumerates E without repetition; (ii) π extends* lo*: $i < j$ implies $\neg(e^j\ \mathsf{lo}\ e^i)$; and (iii) π folds to the state: $e^n(\cdots e^2(e^1(\sigma_0))\cdots) = s$. A configuration is RA-linearizable when every version in the store — not only the replica heads; LCAs are historical versions and the merge reads them — admits a witness.* `IsRALinearizable3`

Why a set-relative order? One definitional correction is forced at the outset: for every internal use, the absorber clause must range over the event set of *the version being linearized*, not over the whole configuration — an event can gain absorbers from another branch, silently cancelling an order edge inside a version whose own state depends on it (the defeater of §3.3 realizes exactly this). We therefore construct witnesses respecting lo^E throughout; since $\mathsf{lo} \subseteq \mathsf{lo}^E$, every such witness also extends the paper's order, and Definition 2.2 is delivered exactly as the paper states it.

Running examples. Three data types exercise every corner of the theory:

- the **counter**: $\Sigma = \mathbb{Z}$, $\mathsf{inc}(\sigma) = \sigma + 1$, $\mathsf{mergeL}(l, a, b) = a + b - l$; all operations commute, but the merge genuinely uses its LCA;
- the **OR-set** (tagged add-wins set): elements are tagged by the adding event's timestamp; rem deletes the tags it has seen; concurrently, $\mathsf{rem} \xrightarrow{\mathsf{rc}} \mathsf{add}$ resolves in favour of the add. The merge keeps an element iff *both* sides have it or *some* side added it since the fork:

  $$\mathsf{mergeL}(l, a, b) \ =\ (a \cap b) \cup (a \setminus l) \cup (b \setminus l)$$

  — pointwise, "if the tag is in l, require it in both branches (nobody deleted it); otherwise one branch suffices (somebody added it)." Here the LCA acts as a *tombstone set*: deletion is represented by *absence relative to l*, so the state itself needs no graveyard;
- the **enable-wins flag**: per-replica pairs (counter, flag); enabling bumps your own counter and sets your flag; disabling clears all flags; the merge compares each replica's counter against the LCA's to decide whether an enable "happened since the fork" on that replica. Its merge *compares* counters rather than accumulating sets — and that difference will be visible in the metatheory (§8).

3 Three facts the metatheory must respect

Before building anything, we record three boundary facts. The first is about the problem: the LCA argument genuinely adds power, so there is no binary shortcut. The second and third are about the paper: its store-level LCA lemma is true but under-proved, and its soundness induction is refuted by a fully legal execution. Together they fix the shape of the corrected metatheory — natively ternary (§3.1), resting on a mechanized store invariant (§3.2), and built around the join rather than the sides (§3.3).

3.1 The LCA argument is load-bearing

The paper notes that CRDTs are the special case "ignore the LCA argument," so binary, CRDT-style reasoning embeds into the ternary setting. One might hope for the converse — collapse the ternary merge to a binary one, verify *that* with lattice-style CRDT techniques, and inherit the result. The converse direction fails, mechanically:

Theorem 3.1 (the counter admits no binary collapse)**.** *The counter satisfies the ternary shared-peel VC*

$$\mathsf{mergeL}(l \cdot e,\ a \cdot e,\ b \cdot e) \ =\ \mathsf{mergeL}(l, a, b) \cdot e$$

— for the counter, $(a{+}1) + (b{+}1) - (l{+}1) = (a{+}b{-}l) + 1$ *— but provably violates the corresponding binary VC* $\mathsf{merge}(a \cdot e, b \cdot e) = \mathsf{merge}(a, b) \cdot e$ *for every LCA-forgetting binary collapse.* `Counter_binary_lem_Oop_false`

So the ternary metatheory cannot be obtained by instantiating a binary one; it must be built natively, with the LCA state threaded through every merge-side statement. Concretely, in the corrected theory the shared peel becomes *three-sided*: a shared event is (by the LCA lemma below) exactly an LCA event, and peeling it removes it from all three components in lock-step.

3.2 The LCA lemma is a reachability invariant — with a gap in the published proof

Everything above silently used:

Lemma 3.2 (LCA lemma)**.** *In every reachable configuration, if* v_ℓ *is an LCA of versions* v_1, v_2 *in the store's DAG, then* $L(v_\ell) = L(v_1) \cap L(v_2)$*.*

This is what lets merge VCs be stated over *event sets*: the state the merge receives as l is the canonical state of exactly the intersection of the two sides' histories. The lemma is true — but it is a *global induction over the reachable store*, not a local fact. Its mechanized proof threads a store invariant (all parents allocated; event sets monotone along edges; and an *origin* invariant — each event appears first at a unique generator version) through every transition; the published proof asserts the generator-uniqueness step without the induction that sustains it. An erratum-sized gap: the statement survives, the proof as written does not. (A related boundary fact: *criss-cross* merges can defeat the *existence* of an LCA — they gate merge-enabledness, not the lemma. Open Question 9.6 takes up the extension.)

3.3 A fully LCA-legal execution defeats the soundness proof

The generic half of the paper's contract is a *bottom-up linearization* theorem: given linearization witnesses for the two sides of a merge, construct one for the merged version by repeatedly *peeling* a final event through mergeL using the catalogue's interchange VCs. One might hope the version DAG's discipline — every merge sitting at an honest LCA — keeps the peel supplied with peelable events. It does not: one staging replica suffices to realize the intersection of two histories as an honest version, and the merge induction is stuck.

Example 3.3 (the defeater)**.** *The data type is a one-key add-wins skeleton with tagged adds:* $\sigma = (A, D) \in 2^{\mathbb{T}} \times 2^{\mathbb{T}}$ *with live set* $A \setminus D$*;* add_t *inserts its tag into* A*;* rem *kills everything it currently sees (*$D \coloneqq A \cup D$ *— the state-dependence is what makes* $\mathsf{add} \not\rightleftharpoons \mathsf{rem}$*); concurrently* $\mathsf{rem} \xrightarrow{\mathsf{rc}} \mathsf{add}$ *(add wins); the merge is componentwise union, ignoring its LCA argument. A legal MRDT. Replica* p *adds* (A_p)*, replica* q *adds* (A_q)*. A staging replica* s *merges both one-add versions, creating* v_s *with* $L(v_s) = \{A_p, A_q\}$*. Now* p *removes* $(R_p$*, seeing only* $A_p)$ *and then merges with* v_s*; symmetrically* q *removes* (R_q) *and merges with* v_s*. Figure 2 shows the DAG. The final merge of the two heads finds* $\mathrm{LCA} = v_s$*, and* $L(v_s) = \{A_p, A_q\} = L(\mathit{head}_p) \cap L(\mathit{head}_q)$ *— the configuration is fully legal. Now count last events.* p*'s head lives on exactly* A_q*'s tag, so a state-correct witness must run* R_p *before* A_q*, and neither* R_p *nor* A_p *can come last: every linearization of* p*'s head ends in* A_q*. Symmetrically, every linearization of* q*'s head ends in* A_p*. Yet in the merged version every tag is dead, so its linearizations must end in one of the removes. The events the induction must peel are buried in the very sides that own them; no ordering of the paper's bottom-up rules applies.*

The two removes commute ($\mathsf{rem} \not\rightleftharpoons \mathsf{add}$ but $\mathsf{rem} \rightleftarrows \mathsf{rem}$), which invites a particular rescue attempt: the paper's BottomUp-2-OP rule (`inter_lca_2op`) admits the commuting case and could, one hopes, peel a remove last. It cannot, for a reason that is a one-line state fact. To peel o last from a side, that side's state must be an o-image — of the form $\mathsf{do}(\sigma', o)$ with o outermost. But in the add-wins skeleton *every* rem-output has empty live set ($\mathsf{rem}(A, D) = (A, A \cup D)$), whereas both heads carry a live tag (head_p lives on A_q, head_q on A_p — the opposite add, merged in *after* the local remove). So neither head is a rem-image, and no bottom-up rule can extract a remove from it. The tempting linearization $[A_p, R_p, A_q, R_q]$ is a valid witness of the *merged* version, but its restriction to head_q folds to the all-dead state, not head_q's actual state (live A_p): it is not assembled from a state-correct side witness, which is exactly what the induction requires. All three facts are mechanized (`InterLca2op_Defeater_Arbiter`: `awset_rem_output_empty`, `no_inter_lca_2op_rem_peel_of_defeater`, `crack1_witness`).

The consequence for the design: the metatheorem cannot be patched peel-by-peel — it must be rebuilt on canonical states and a Join Lemma, with merge-side VCs that are statements *about the join*, not about how either side's state factors.

4 Canonical states and the ternary Join Lemma

We now rebuild — new proof architecture, and with it a *new set of verification conditions* to replace the paper's catalogue. The lesson of the defeater is architectural: witnesses for a merged version cannot be manufactured out of witnesses for its sides, so the corrected metatheory must not traffic in witness lists at all. The rebuild has two layers. The *update layer* makes states, not sequences, the objects of the induction: it assigns to every event set E a single well-defined state, its *canonical state* (Definition 4.2 below). The *merge layer* is then one equation between

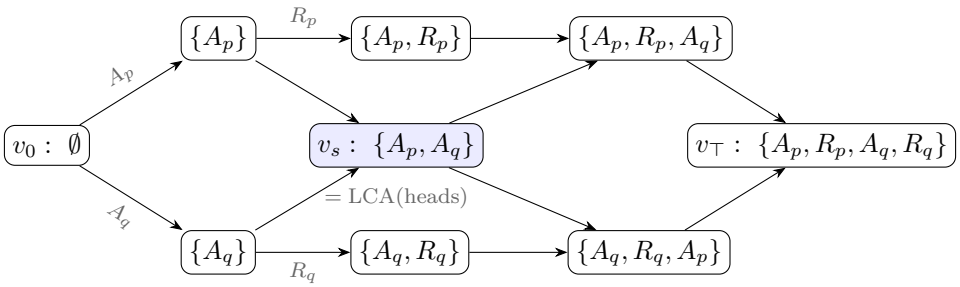

Figure 2: The defeater (Example 3.3). Nodes are versions labelled by their event sets; the shaded version v_s, created by a staging replica, realizes $L(\text{head}_p) \cap L(\text{head}_q)$ as an honest store version, so the final merge is LCA-legal. The removes R_p, R_q — the only events a bottom-up derivation may peel last — are buried mid-history on their own sides.

canonical states — the ternary *Join Lemma* — and establishing it from per-data-type VCs becomes the entire burden of the metatheorem (§5–§6).

The update layer comes first, and a structural observation makes it cheap: this half of the corrected theory never mentions the merge. Its one theorem is stated over the update signature $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{rc} \rangle$ alone:

Theorem 4.1 (convergence)**.** *Assume the update-layer VCs (items 1–3 of §6). Then any two* lo^E*-respecting enumerations of a finite event set* E *fold from* σ_0 *to the same state.*

This is where the set-relative reading of §2 earns its keep: with the paper's configuration-wide order in place of lo^E, the theorem is *false* — the defeater's heads are already counterexamples, since two order-respecting replays of the three events of head_p reach different states. Convergence licenses a definition:

Definition 4.2 (canonical state)**.** *For a finite event set* E*, the canonical state* $\sigma(E)$ *is the state obtained by folding any* lo^E*-respecting enumeration of* E *from* σ_0*. At least one respecting enumeration exists because* lo^E *is acyclic (VC 2); the result is independent of the choice by Theorem 4.1; and* $\sigma(\emptyset) = \sigma_0$*. We write* $\sigma(E)$ *for the canonical state of an event set, and* σ_v *for the state a version* v *happens to hold in the store — proving that the two coincide is precisely the metatheorem's job.*

With canonical states in hand, witness lists disappear as promised: a version linearizes iff $\sigma_v = \sigma(L(v))$ — the witness, when one is owed, is any respecting enumeration of $L(v)$. The induction can therefore maintain a state-level invariant ("every version holds σ of its events") instead of constructing sequences. The one genuinely ternary statement is what the merge must produce:

Definition 4.3 (ternary Join Lemma)**.** *For backward-closed event sets* E_1, E_2 *of a reachable configuration,*

$$\sigma(E_1 \cup E_2) \;=\; \mathsf{mergeL}\big(\sigma(E_1 \cap E_2),\ \sigma(E_1),\ \sigma(E_2)\big).$$

By Lemma 3.2 the first argument is exactly the state the store hands the merge. The adequacy proof (§7) is then an induction over the transition system maintaining "every version holds the canonical state of its event set"; the merge case *is* the Join Lemma. What remains is the question this note exists to answer: which per-data-type VCs make the Join Lemma true?

5 Why the contract is a delta contract

Where should the laws that prove the Join Lemma come from? The obvious supplier is the CRDT tradition. For state-based CRDTs — binary merge, no LCA — merge-coherence is lattice-flavoured: associativity, idempotence, inflation of updates. Each of the three provably fails under ternary merge, and this section works through the failures, because the shape of what fails is what reveals the contract that succeeds.

No idempotence. The *diagonal* law $\mathsf{mergeL}(a, a, a) = a$ — the paper's own MERGEIDEMPOTENCE — does hold (for the counter: $a+a-a = a$), but it is not the law a lattice-style proof consumes. The binary architecture uses idempotence to absorb a duplicated *side*, $\mathsf{merge}(a, a) = a$, with nothing tying the two copies to a common past; the ternary analogue is $\mathsf{mergeL}(l, a, a) = a$ with l *free*, and for the counter $\mathsf{mergeL}(l, a, a) = 2a-l$: false whenever $l \neq a$. Only feasible triples are idempotent — and feasibility (equal sides share their whole history, so l is the canonical state of the intersection) forces $l = a$, collapsing the law back to the diagonal. Unconditional side-idempotence would exclude the counter, i.e. exclude precisely the data types the ternary theorem exists for.

No usable order. The lattice-style workhorse order $x \sqsubseteq y \coloneqq \mathsf{merge}(x, y) = y$ has ternary candidates $x \sqsubseteq_l y \coloneqq \mathsf{mergeL}(l, x, y) = y$, but for the counter this relation degenerates to $x = l$; no antisymmetry survives to power order-theoretic reasoning. The single contextual VC below is accordingly an *equation*, not an inequality.

Associativity is the wrong target. The natural guess — $\mathsf{mergeL}(l, \mathsf{mergeL}(l, a, b), c) = \mathsf{mergeL}(l, a, \mathsf{mergeL}(l, b, c))$ — assumes one LCA for all three merges. In an execution, re-associating a three-way reconciliation involves *four* distinct LCAs:

$$\mathsf{mergeL}\big(l_{(ab)c},\, \mathsf{mergeL}(l_{ab}, a, b),\, c\big) \;\stackrel{?}{=}\; \mathsf{mergeL}\big(l_{a(bc)},\, a,\, \mathsf{mergeL}(l_{bc}, b, c)\big),$$

with $l_{ab} = \sigma(E_a \cap E_b)$, $l_{(ab)c} = \sigma((E_a \cup E_b) \cap E_c)$, and so on. For the counter — whose canonical states are additive measures over event sets — the two sides come to $a + b + c$ minus the two LCA measures, and they agree because

$$(E_a \cup E_b) \cap E_c = (E_a \cap E_c) \cup (E_b \cap E_c) \quad\Longrightarrow\quad \text{both LCA-sums} = |E_{ab}| + |E_{ac}| + |E_{bc}| - |E_{abc}|$$

by inclusion–exclusion. The law holds, but only *through the set-algebra of intersections*: over four unconstrained states it is false. Honest associativity is inherently a feasible-tuple fact — and, the mechanization's pleasant surprise, **the corrected induction never needs it**. Every merge the induction forms already sits at its honest LCA; the only laws consumed are two *redistribution* identities that equate two honestly-formed merges:

Definition 5.1 (the delta contract)**.**

$$\begin{aligned}
\textit{LOCAL-REDISTRIBUTE}: \quad & \mathsf{mergeL}(l,\, \mathsf{mergeL}(m, x, c),\, y) \;=\; \mathsf{mergeL}(m,\, \mathsf{mergeL}(l, x, y),\, c), \\
\textit{REDISTRIBUTE}: \quad & \mathsf{mergeL}\big(\mathsf{mergeL}(m, x_0, c),\, \mathsf{mergeL}(m, x_1, c),\, \mathsf{mergeL}(m, x_2, c)\big) \\
& \qquad\qquad = \;\mathsf{mergeL}\big(m,\, \mathsf{mergeL}(x_0, x_1, x_2),\, c\big).
\end{aligned}$$

Read c as a *delta relative to* m (in the induction, c will be "apply event e to its causal past"). LOCAL-REDISTRIBUTE says a delta application commutes past an enclosing merge through one branch slot. REDISTRIBUTE says a delta carried by *all three* components extracts *once* — the LCA slot cancels the duplicates by arithmetic. This is the delta contract's quiet coup: cancelling the duplicated shared contribution is exactly the job idempotence performs in lattice-based convergence arguments, now done without any idempotence — which is why the counter sails through (REDISTRIBUTE for the counter is the identity $\sum x_i + 3c - 3m - (x_0 + c - m) - \cdots$ collapsing on both sides).

Where the contract holds unconditionally — and where it cannot. The two laws hold *on all states* exactly for the group-like class (the counter) and the lattice-like, LCA-blind class (grow-only structures). They are *unconditionally false* for the OR-set: take an element tag present in c and l but in neither branch — the two sides of LOCAL-REDISTRIBUTE then disagree about whether the LCA's copy survives. But such a tuple can never arise: a tag in a canonical LCA state is in *both* branches unless removed, and a removal is itself an event of the branch. The failing tuples are *infeasible*, and this is a theorem-grade phenomenon, not an inconvenience:

Finding. *For LCA-sensitive MRDTs beyond the group $\oplus$ lattice classes, the merge-coherence laws are irreducibly feasibility-conditioned: true on canonical tuples at honest LCAs, false in general. Quantifying merge VCs over all states — the published catalogue's style — is not merely inconvenient for such data types; for several laws it is wrong in principle. (The enable-wins flag even falsifies the unit law* $\mathsf{mergeL}(\sigma_0, \sigma_0, s) = s$ *on an infeasible state with a set flag and a zero counter.)*

Accordingly the delta laws enter the final contract in feasibility-conditioned form: quantified over canonical states of backward-closed event sets, with every mergeL node at its honest LCA. The unconditional form remains available as a one-line *on-ramp* (it trivially implies the conditioned form) for the group and lattice classes.

6 The eight verification conditions

The update-side obligations of §4 and the merge-side laws of §5 now assemble into the full contract — a *new* VC set, replacing the paper's catalogue of twenty-four. For an MRDT $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc} \rangle$, the contract is:

Update layer (unconditional, SMT-shaped).

1. **rc-non-comm (guarded)** — for distinct events *from different replicas*: they fail to commute iff rc orders them in exactly one direction. *The conflict table is the commutativity table, with a direction.* The different-replica guard is a semantic necessity, not a convenience: same-replica events are totally ordered by program order, so they are never concurrent and there is no conflict for rc to resolve. An unguarded VC would instead demand that rc order *every* non-commuting pair — unsatisfiable for real data types: the *optimized OR-set*'s same-replica same-element adds do not commute (the newer add evicts the older tag), yet they can never race, and its rc rightly ignores them. (The paper's F* artifact carries exactly this guard in its interface — `App_mrdt.fsti`: `requires distinct_ops o1 o2 /\ get_rid o1 <> get_rid o2`.)

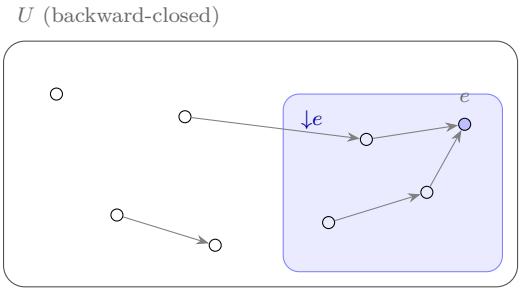

Figure 3: The causal-delta equation. The shaded region is e's causal past $\downarrow e$ (what e saw); e is lo^U-maximal. e's effect on the whole world A equals: compute e's effect on its own causal past B, then merge that delta into A *over LCA* B. The merge sits at its honest LCA: $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$. Concurrent context that e never observed cannot amplify what e does.

2. **no-rc-chain** — conflict priorities do not chain: never $o_1 \xrightarrow{\mathsf{rc}} o_2$ and $o_2 \xrightarrow{\mathsf{rc}} o_3$. This is what makes the linearization order acyclic, so maximal events exist — the entire peel-and-recurse architecture stands on it.

3. **cond-comm** — a resolved conflict is invisible once an absorber runs: swapping an rc-ordered concurrent pair deep in a fold changes nothing for any later non-commuting event, across arbitrary intervening events. This is the engine of convergence.

Merge layer.

4. **merge symmetry** — $\mathsf{mergeL}(l, a, b) = \mathsf{mergeL}(l, b, a)$.

5. **feasible unit** — $\mathsf{mergeL}(\sigma_0, \sigma_0, \sigma(E)) = \sigma(E)$: merging over nothing is a no-op, on states that can arise.

6. **feasible local-redistribute** and

7. **feasible redistribute** — Definition 5.1, quantified over canonical tuples at honest LCAs.

8. **the causal-delta equation** (CDVC3) — for a lo^U-maximal event e of a backward-closed U, with $A = \sigma(U \setminus \{e\})$ and $B = \sigma(\downarrow e \setminus \{e\})$ the canonical state of e's own causal past:

$$\mathsf{mergeL}\big(B,\ A,\ e(B)\big)\ =\ e(A).$$

   What an event does is determined by what it saw (Figure 3): applying e to the whole world equals computing e's delta on its causal past and merging it in over that past.

Items 1–4 are one-liners for every data type we have discharged; items 6–7 frequently hold unconditionally anyway (for the OR-set, REDISTRIBUTE is a pointwise Boolean tautology); essentially the entire per-data-type proof effort concentrates in item 8, whose discharge follows a uniform recipe: characterize $\sigma(E)$ (a "sandwich" invariant along canonical enumerations), then a *trichotomy* placing every event the maximal e interacts with either inside $\downarrow e$ or absorbed on a side that owns it.

Compare the published catalogue: 24 VCs per data type, plus five more the soundness proof uses silently. Seventeen of the 24 — the BottomUp induction-scheme families — exist to drive the broken induction and are consumed nowhere in the corrected one; the paper's MergeIdempotence is likewise never used. The seventeen were, however, doing honest work of a different kind: they are an *automation layer*, Skolemizing "holds on feasible states" into base-and-step equations an SMT solver can discharge. Rebuilding that layer, aimed at the corrected target (CDVC3 rather than BottomUp), is in our view the most valuable open engineering problem this work leaves (§9).

7 Adequacy

The contract delivers. Everything a data type owes is the eight equations of §6; everything else is proved once, for all data types, as part of the execution model.

Theorem 7.1 (soundness of the eight VCs)**.** *If an MRDT satisfies VCs 1–8, then every configuration reachable from the initial configuration in the ternary transition system is RA-linearizable, per version.* `ra_linearizable_of_core_feasible_cd3`

The proof shape: the update layer powers the set-relative convergence theorem, hence canonical states; the transition induction maintains "every store version is canonical for its event set" (fork and query are trivial; apply is the extend lemma); the merge case reduces, via Lemma 3.2, to the ternary Join Lemma, which is proved from VCs 4–8 by strong induction on $|E_1 \cup E_2|$: select a $\mathsf{lo}^{E_1 \cup E_2}$-maximal e, decompose each side containing e through $\downarrow e$ (local-redistribute at the honest LCA $\downarrow e \setminus \{e\}$), cancel the shared delta (redistribute), close the principal step with CDVC3, and recurse. The buried-event pathology of Example 3.3 dissolves structurally: no step ever asks how a *side's* state factors — every factoring happens through the join.

Everything else one might fear owing is proved once, for all data types, as an execution-model invariant: visibility transitivity, per-version backward closure, store well-formedness, and Lemma 3.2 itself. The data type owes exactly the eight equations.

A second route, and an honest asymmetry. One discharged data type does not fit the pattern above. The enable-wins flag's merge *compares counters* against the LCA, and for such merges the Join Lemma's hypotheses as stated (backward closure under non-commuting-visibility) are provably too weak: there is a closure-legal tuple — a live LCA enable, a dead post-LCA enable inflating one side's counter, the killing disable visible only to the other side — on which the production merge computes the wrong flag. Full *causal* closure (which the store invariant actually supplies) excludes it, and the flag is discharged through a full-closure variant of the join. The finding: **required closure strength is a function of the merge's shape** — commutation-closure suffices for set-shaped merges, counter-comparison merges need full causal closure. Reunifying the two routes was expected to be routine — "full closure only strengthens what a data type may assume." Mechanization shows it is not (§9, Open Question 9.3): the naive reunification — re-run the Join Lemma induction with full-closure hypotheses — is refuted, because no single-event (or block) peel preserves full closure. What survives is a *closure-indexed* contract in which the data type *declares* its closure strength; the soundness bridge is uniform across strengths, so no reunification into one strength is needed.

MRDT	contract tier	end-to-end theorem	σ-characterization (one line)
Grow-only set	unconditional	`goset_ra_linearizable3`	union of adds
Grow-only map	unconditional	`gomap_ra_linearizable3`	union of (k, v) records
Incr.-only counter	unconditional	`ioc_ra_linearizable3`	#events
PN-counter	unconditional	`pn_ra_linearizable3`	#inc − #dec
RGA (tombstone)	unconditional	`rga_ra_linearizable3`	grow-only nodes + tombstones
Peritext	unconditional	`peritext_ra_linearizable3`	grow-only mark records
OR-set	feasible	`ORSet_ra_linearizable3`	tag live iff added, no vis-later rem
OR-set (efficient)	feasible	`ORSetE_ra_linearizable3`	… or same-replica add later (eviction)
Enable-wins flag	full-closure	`EWFlag_ra_linearizable3`	per key: ctr = #enables; flag iff unkilled enable
Multi-valued register	feasible	recipe (trichotomy-free: all-comm ⇒ $B{=}\sigma_0$)	
Add-wins priority queue	feasible	recipe (the OR-set pattern on the add component)	
RGA (tombstone-free)	*blocked*	feasible *update* layer needed (below)	

Table 1: The Sal production catalogue against the eight VCs. Nine of twelve carry kernel-checked end-to-end RA-linearizability; all discharges live in a single `MRDT_Instances.lean`.

8 The catalogue, discharged

Table 1 is the empirical heart of the note: the VC set is not merely adequate in principle — the data types people actually ship satisfy it. Three lessons from the sweep:

Tombstones are a metatheoretic trivializer. The catalogue's celebrated hard cases — the RGA sequence type and the Peritext rich-text type — land in the *easiest* tier. The reason is almost embarrassing: with tombstones, deletion *adds* a record; every state component is grow-only; all operations commute; the merge is a blind join. All of the RGA's genuine difficulty lives in the *read-side* traversal (turning the node soup into a sequence), which RA-linearizability of the state never touches. By contrast the *tombstone-free* RGA — which physically deletes and repairs positions by climbing ancestor paths — is the one data type the corrected theory cannot yet host, for a reason worth stating precisely: its commutation VC is itself *reachability-conditioned* (two inserts commute only on well-formed states), and our update layer, like the paper's, quantifies over all states. The missing piece is a feasible *update* layer — update VCs conditioned on an update-and-merge-closed invariant, plus a transport lemma showing every state a canonical enumeration folds through satisfies it. That is the theory's sharpest open edge, and precisely the question the tombstone-free RGA was built to force.

Commutation class and delta class are independent axes. The multi-valued register is all-commuting, yet *not* in the unconditional tier: its merge is intersection-shaped, $(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$, and fails the delta laws on infeasible tuples exactly as the OR-set does. The boundary between tiers is drawn by *how the merge uses its LCA*, not by whether operations commute. (The compensation: all-commuting makes every punctured causal past empty, $B = \sigma_0$, so the feasible discharge needs no trichotomy at all.)

The eight VCs certify real code on its historical fault line. The enable-wins flag's repository history contains a known-broken sibling: a global-counter variant whose compositional VC (`inter_right_1op`) fails on DAG-shaped executions, the very defect per-replica counters were introduced to fix. The mechanized σ-facts prove the production merge computes exactly union-liveness — *verified on precisely the corner where the sibling fails.* This is the methodology

working as intended: the metatheory does not merely bless the catalogue, it explains *why* the fixed design is right.

9 Open questions

The mechanization settles what the paper claimed; it also sharpens what remains genuinely open. Six questions, roughly in order of theoretical depth:

Open Question 9.1 (CDVC3-derivability)**.** *Is* CDVC3 *derivable from VCs 1–7 (plus the unconditional delta laws where they hold)? Specializing the merge to an LCA-blind lattice join — the CRDT case — the question closes exactly: there, the corresponding causal-delta bound is equivalent to the Join Lemma and no strictly weaker bridge VC exists, with both attack routes characterized — the positive induction is circular at two identified case shapes, and the countermodel space is narrowed to non-atomic lattice states (all of this mechanized, in* `Sal/CRDTs/Metatheory/`*). The ternary question is open.*

Open Question 9.2 (structure theorem)**.** *Empirically, the unconditional delta contract holds exactly on the group and lattice classes. Is every unconditional-*DeltaVCs3 *merge a torsor-like group* $\otimes$ *lattice combination? (The naïve two-sorted umbrella* $\mathsf{mergeL}(l, a, b) = a \boxplus (b \boxminus l)$ *does not derive the contract uniformly, so the question is genuinely open.)*

Open Question 9.3 (route reunification — naive route refuted, closure-indexed resolution mechanized)**.** *The obvious reunification — restate the delta laws and* CDVC3 *over full causal closure and re-run the Join Lemma induction — fails, and the failure is mechanized. The induction must peel an event that is both* lo^U*-maximal (placeable last) and vis-maximal (its removal keeps the sides fully closed); on a reachable four-event configuration these two maximal sets are disjoint, and the obstruction persists for every block peel, in both directions (*`JoinLemma3C: no_proper_back_block, no_proper_front_block`*). A wider induction class (fully-closed minus a* lo*-upward peel set) moves the obstruction but does not remove it: individually fully-closed sides admit no common witness set to decompose against (*`killTest_no_common_U`*). What does work needs no reunification: a* ***closure-indexed contract*** JoinLemma3C $\mathcal{C}$ *in which the data type declares its closure strength* $\mathcal{C}$*, with a single soundness bridge (*`ra_linearizable3_of_joinC`*): store versions are fully causally closed, so* JoinLemma3C $\mathcal{C}$ *applies for any* $\mathcal{C}$ *that full closure implies — both weak and full qualify, and the whole* `GoodConfig3` *induction is reused verbatim. The two routes of §7 are the two instances, and all nine production discharges route through the bridge unchanged (*`AllMRDTs_viaContract`*), reusing their existing VCs verbatim: eight at* $(\mathit{weak}, \top)$ *— the OR-sets via the feasible triple, the six unconditional types via the delta on-ramp — and the enable-wins flag at* $(\mathit{full}, \top)$*; the disjunction is the contract, not a defect to be reunified. This is the "no new mathematics" the resolution promised — confirmed, with one correction: the bridge holds exactly for* $\mathcal{C}$ *weaker than full closure (a* $\mathcal{C}$ *stronger than full makes* JoinLemma3C *too weak to be adequate). Genuinely open: whether the single-strength contract (carrying the common witness set as data, re-founding* CDVC3 *on the full downset) buys anything over the closure-indexed form.*

Open Question 9.4 (the feasible update layer — naive conditioning refuted, feasibility notion sharpened)**.** *Condition the update-layer VCs on an update-and-merge-closed invariant and host the tombstone-free RGA. The required invariant machinery (*RgaInv $\wedge$ id_mono*, closed under merge given id-monotone allocation) is already mechanized on the data-type side — but the*

conditioning of convergence itself is subtler than "restrict the VCs to Inv*-states," and the naive form is refuted (`G2_Transport_Probe`). Replacing $\not\rightleftarrows$ by its* Inv*-and-applicability-conditioned form inside* lo^E *drops order edges between events that are never jointly applicable (an* Ins *and a* Del *of the same node vacuously "commute"), and two orders that fold to different states then both respect* lo^E*: convergence fails. Strengthening* Inv *cannot repair it — conditioning is antitone, so a smaller applicability domain only removes more edges. The order must instead be applicability-aware: an edge survives when one event's applicability depends on the other. Restricting convergence to strictly applicability-valid enumerations is not the answer, though it looks strictly more general (`G2_Applicability_Aware: separating_inequivalence`): it is unsatisfiable for genuinely reachable versions carrying redundant concurrent ops — two concurrent deletes of one node, for which no enumeration keeps both deletes applicable. The resolution, and the finding, is that the two repairs are complementary, not competing: the correct notion is the applicability-aware order together with no-op-feasible enumerations (each step applicable or acting as identity). The order excludes the dependency-violating replays (which mere no-op tolerance would re-admit, breaking convergence); no-op tolerance admits the harmless redundant ones (which strict applicability rejects, breaking satisfiability). On the two decisive cases — one dependency, one redundancy — this notion is provably both convergent and satisfiable, where each single repair fails one (`UpdateFeasibility_Gate: loOnA_noopFeasible_verdict`). What remains is the general convergence theorem over this notion, and the RGA discharge on top of it.*

Open Question 9.5 (the automation layer)**.** *The published catalogue's seventeen induction-scheme VCs were an SMT-facing Skolemization of feasibility, aimed at the wrong target. Design the analogous per-data-type induction scheme whose induction implies* CDVC3 *and the feasible delta laws — restoring push-button discharge, now of VCs that actually imply RA-linearizability. The observed uniformity of the discharges (σ-characterization by sandwich invariant + absorber trichotomy) suggests the scheme exists.*

Open Question 9.6 (criss-cross merges and virtual LCAs)**.** *The transition system gates Merge on the existence of an LCA (§3.2); criss-cross configurations are reachable, and every version in them linearizes, but their heads cannot merge — the model is silent exactly where git reaches for its recursive strategy (merge the maximal common ancestors into a virtual base, then merge with that). The metatheory looks ready for the extension: the ternary Join Lemma is stated over backward-closed event sets, and $E_1 \cap E_2$ needs no witnessing store version. What remains is a virtual-LCA Merge rule plus one per-data-type question: do the eight VCs imply that the recursively computed base equals $\sigma(E_1 \cap E_2)$? For the counter this is inclusion–exclusion once more — exact iff the maximal common ancestors jointly cover the intersection, itself a candidate store invariant; for the OR-set, a pointwise Boolean check. A positive answer closes the one visible gap between the model and practice.*

10 Mechanization

Everything above is mechanized in Lean 4 (Mathlib; axioms: `propext`, `Classical.choice`, `Quot.sound`; no `sorryAx` anywhere) in the Sal repository, under `Sal/MRDTs/Metatheory/`:

Artifact	Lean	File
signature, ternary lo	`MRDTSig`, `ConditionedMRDTSig`	`MRDTSig.lean`
store, DAG, transitions	`Configuration`, `Step3`	`ExecutionModel.lean`, `LCA_Lemma.lean`
Lemma 3.2	`lca_events_of_storeInv`	`LCA_Lemma.lean`
σ/lo^E layer (merge-independent)	`UpdateVCs`, `IsCanonicalState`	`Sigma_LoOn3.lean`
the eight VCs	CoreVCs3CD, FeasibleDeltaVCs3, CDVC3	`VC_Set.lean`
Theorem 7.1 + bridges	`ra_linearizable_of_core_feasible_cd3`	`Adequacy.lean`
Table 1	all instance theorems	`MRDT_Instances.lean`
Theorem 3.1, kill-tests	—	`Development/Impossibility.lean`
findings journal T0–T11	—	`Development/MRDT_METATHEORY_DRAFT.md`
Conditioned-metatheory investigation (§3.3, Open Questions 9.3 & feasible-update):		
defeater vs. 2-OP (§3.3)	`no_inter_lca_2op_rem_peel_of_defeater`	`Development/InterLca2op_Defeater_Arbiter.lean`
peel obstruction (OQ 9.3)	`reunification_peel_obstruction`, `no_proper_back_block`	`Development/{Reunification_Peel_Obstruction,JoinLemma3C}.lean`
closure-indexed contract	`JoinLemma3C`, `killTest_no_common_U`	`Development/JoinLemma3F_Of_AlmostClosed.lean`
closure-indexed adequacy + instances	`ra_linearizable3_of_joinC`, `ConditionedContract`	`Development/ConditionedContract.lean`
all 9 discharges via the contract	`*_adequate_viaContract` (9)	`Development/AllMRDTs_viaContract.lean`
update-layer feasibility notion	`loOnA_noopFeasible_verdict`	`Development/UpdateFeasibility_Gate.lean`
feasible-update gate	`G2_conditioned_convergence_refuted`, `separating_inequivalence`	`Development/G2_{Transport_Probe,Applicability_Aware}.lean`

The update-side layer (lo^E, convergence, σ) is merge-independent — it is stated over the update signature $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{rc} \rangle$ alone — and is shared with the binary (CRDT) specialization of the theory; in the repository it lives in `Sal/CRDTs/Metatheory/`, the directory that also holds the binary exactness results quoted in Open Question 9.1. The production data types of Table 1 are mirrored faithfully from `Sal/MRDTs/*` (encoding deviations documented per instance); their original 24-VC discharges are untouched.

The conditioned-metatheory artifacts under `Development/` are exploratory — they establish the refutations and the closure-indexed resolution cited in §3.3 and the open questions, and each cited theorem is kernel-checked with axioms in {`propext`, `Classical.choice`, `Quot.sound`} (no `sorryAx`). They import one linearization file carrying two unrelated `sorry`s that the cited theorems do not transitively depend on; `Development/CONDITIONED_METATHEORY_PLAN.md` is their roadmap.